



INDICADOR DE LOGRO IDL3

Big Data Aplicada

Elaborado por:

APAZA PEREZ OSCAR GONZALO
CABEZAS HUANIO RUBEN KELVIN
RUIZ ALVA JERSON ENMANUEL
PONCE DE LEON TORRES FABYOLA KORAYMA

Solicitado por:

ORIZANO SALVADOR SERGIO VICTOR

Huancayo, 2025

Tabla de contenidos

1	Verificando Hadoop	2
2	Verificando Spark	3
3	Subiendo sales_data.csv a hdfs	4
4	Iniciando la sesion de Spark	4
5	Configurar un esquema NoSQL	5
6	Benchmark MongoDB	8

1 Verificando Hadoop

```
!hadoop version
# start-all.sh desde usuario hadoop
# jps
```

Hadoop 3.4.1

Source code repository <https://github.com/apache/hadoop.git> -r 4d7825309348956336b8f06a08322b78422

Compiled by mthakur on 2024-10-09T14:57Z

Compiled on platform linux-x86_64

Compiled with protoc 3.23.4

From source with checksum 7292fe9dba5e2e44e3a9f763fce3e680

This command was run using /home/hadoop/hadoop/share/hadoop/common/hadoop-common-3.4.1.jar

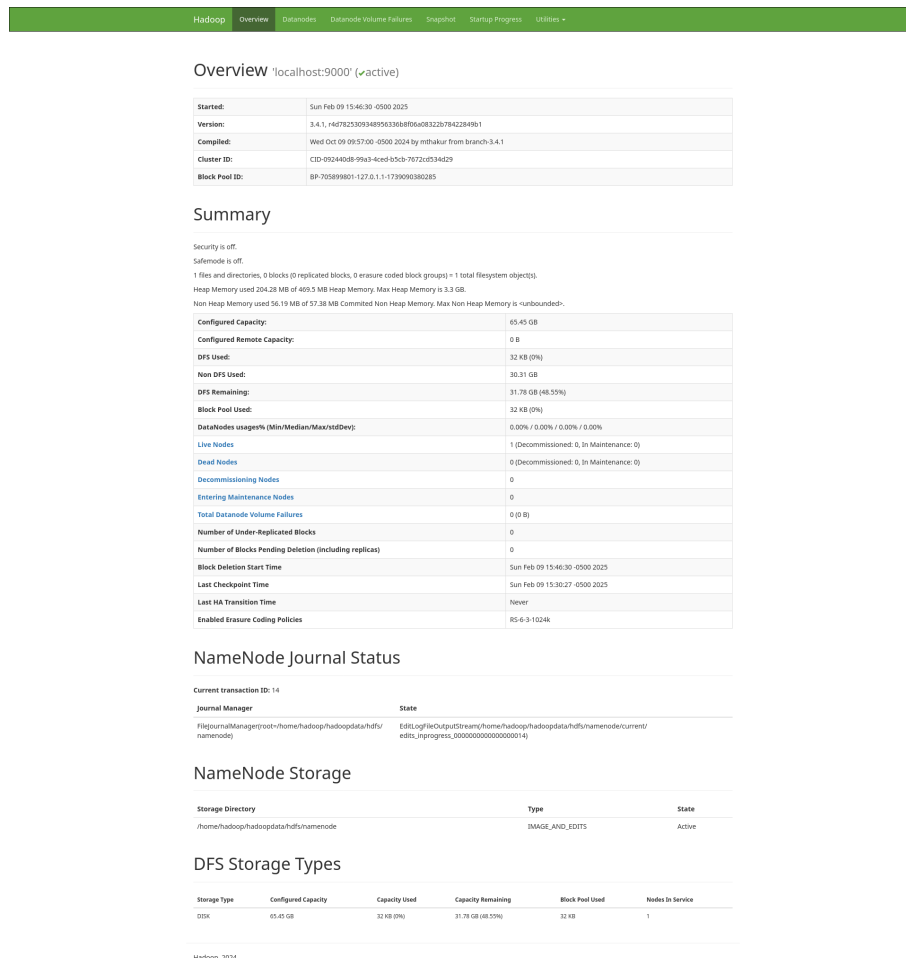
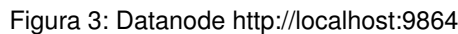


Figura 1: Namenode <http://localhost:9870/>



```
!spark-shell --version
# start-master.sh desde usuario hadoop
# jps | grep Master
```

```

  /---/
 \  \  /---\  /---/
 /---/ .---\ , /---/ /---\
 /---/

```

version 3.5.4

Oficina de Gestión de la Calidad

Spark 3.5.4 **Spark Master at spark://ubuntu:7077**

URL: spark://ubuntu:7077
 Alive Workers: 0
 Cores in use: 0 Total: 0 Used
 Memory in use: 0.0 B Total: 0.0 B Used
 Resources in use:
 Applications: 0 Running, 0 Completed
 Drivers: 0 Running, 0 Completed
 Status: ALIVE

Workers (0)

Worker Id	Address	State	Cores	Memory	Resources
No workers are currently running.					

Running Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
No running applications.								

Completed Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
No completed applications.								

Figura 4: Spark Master at spark://ubuntu:7077

3 Subiendo sales_data.csv a hdfs

```
# En el usuario hadoop
# sudo cp /home/jerson/Documentos/quarto/sales_data.csv /home/hadoop/hadoopdata/
# hdfs dfs -mkdir /sales
# hdfs dfs -put /home/hadoop/hadoopdata/sales_data.csv /sales
!hadoop fs -ls /sales
```

```
Found 1 items
-rw-r--r-- 1 hadoop supergroup      31219 2025-02-09 17:20 /sales/sales_data.csv
```

4 Iniciando la sesion de Spark

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import sum as spark_sum

def create_spark_session():
    """
    Crea una sesión de Spark configurada para trabajar con HDFS
    """
    spark = SparkSession.builder \
        .appName("HDFSProcessing") \
        .config("spark.driver.host", "192.168.18.80") \
        .config("spark.driver.bindAddress", "192.168.18.80") \
        .config("spark.hadoop.fs.defaultFS", "hdfs://localhost:9000") \
        .config("spark.executor.memory", "2g") \
        .config("spark.driver.memory", "2g") \
        .getOrCreate()

    spark.sparkContext.setLogLevel("ERROR")
    return spark

def read_and_process_sales():
    try:
        # Crear sesión Spark
        spark = create_spark_session()

        # Ruta del archivo en HDFS
        hdfs_path = "hdfs:///sales/sales_data.csv"
```

```
# Leer el archivo CSV desde HDFS
df = spark.read.option("header", "true") \
               .option("inferSchema", "true") \
               .csv(hdfs_path)

# Realizar el análisis
sales_by_region = df.groupBy("Region") \
                    .agg(spark_sum("Sales").alias("TotalSales")) \
                    .orderBy("Region")

# Mostrar resultados
print("Ventas por región:")
sales_by_region.show()

return sales_by_region

except Exception as e:
    print(f"Error en el procesamiento: {str(e)}")
    raise
finally:
    spark.stop()

if __name__ == "__main__":
    read_and_process_sales()
```

Setting default log level to "WARN".

To adjust logging level use `sc.setLogLevel(newLevel)`. For SparkR, use `setLogLevel(newLevel)`.

25/02/11 20:22:04 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform...

Ventas por región:

```
+-----+-----+
|Region|TotalSales|
+-----+-----+
| East|    296780|
| North|    294153|
| South|    351882|
| West|    313536|
+-----+-----+
```

5 Configurar un esquema NoSQL

```
from pyspark.sql import SparkSession
from pymongo import MongoClient
from datetime import datetime
import pandas as pd

# !sudo systemctl start mongod desde usuario hadoop

def create_spark_session():
    """Crear sesión Spark para leer datos"""
    return SparkSession.builder \
        .appName("SalesETL") \
        .config("spark.driver.host", "192.168.18.80") \
        .config("spark.driver.bindAddress", "192.168.18.80") \
        .config("spark.hadoop.fs.defaultFS", "hdfs://localhost:9000") \
        .getOrCreate()

def date_to_datetime(date_obj):
```

```
"""Convierte un objeto date a datetime"""
if isinstance(date_obj, datetime):
    return date_obj
return datetime.combine(date_obj, datetime.min.time())

def process_and_load_to_mongo():
    # Configuración MongoDB
    client = MongoClient('mongodb://localhost:27017/')
    db = client['sales_db']

    # Limpiar colecciones existentes
    db.sales.drop()
    db.region_stats.drop()
    db.product_stats.drop()

    # Crear colecciones con índices optimizados
    sales_collection = db['sales']
    region_stats = db['region_stats']
    product_stats = db['product_stats']

    # Crear índices para búsquedas rápidas
    sales_collection.create_index([("region", 1)])
    sales_collection.create_index([("product", 1)])
    sales_collection.create_index([("date", 1)])

    try:
        # Leer datos con Spark
        spark = create_spark_session()
        df = spark.read.csv("hdfs:///sales/sales_data.csv", header=True, inferSchema=True)

        # Convertir a pandas para procesamiento
        pandas_df = df.toPandas()

        # Procesar y cargar ventas individuales
        for _, row in pandas_df.iterrows():
            # Convertir la fecha a datetime
            date_obj = date_to_datetime(row['Date'])

            sale_doc = {
                "region": row['Region'],
                "product": row['Product'],
                "date": date_obj,
                "sales": float(row['Sales']),
                "year_month": date_obj.strftime('%Y-%m')
            }
            sales_collection.insert_one(sale_doc)

        # Agregaciones por región
        pipeline_region = [
            {"$group": {
                "_id": "$region",
                "total_sales": {"$sum": "$sales"},
                "avg_sales": {"$avg": "$sales"},
                "sales_count": {"$sum": 1},
                "products": {"$addToSet": "$product"}
            }}
        ]
        region_results = list(sales_collection.aggregate(pipeline_region))
```

```
region_stats.insert_many(region_results)

# Agregaciones por producto
pipeline_product = [
    {"$group": {
        "_id": "$product",
        "total_sales": {"$sum": "$sales"},
        "regions_sold": {"$addToSet": "$region"},
        "first_sale": {"$min": "$date"},
        "last_sale": {"$max": "$date"}
    }}
]
product_results = list(sales_collection.aggregate(pipeline_product))
product_stats.insert_many(product_results)

return client

except Exception as e:
    print(f"Error específico durante el procesamiento: {type(e).__name__} - {str(e)}")
    raise

def print_collection_contents(client):
    """Función para verificar los datos cargados"""
    db = client['sales_db']

    print("\nContenido de la colección 'sales':")
    for doc in db.sales.find().limit(3):
        print(doc)

    print("\nEstadísticas por región:")
    for doc in db.region_stats.find():
        print(doc)

    print("\nEstadísticas por producto:")
    for doc in db.product_stats.find():
        print(doc)

def verify_data_types(client):
    """Verificar los tipos de datos en la colección"""
    db = client['sales_db']
    first_doc = db.sales.find_one()
    if first_doc:
        print("\nTipos de datos en el primer documento:")
        for key, value in first_doc.items():
            print(f"{key}: {type(value)}")

if __name__ == "__main__":
    try:
        client = process_and_load_to_mongo()
        print("Datos cargados exitosamente")
        print_collection_contents(client)
        verify_data_types(client)
    except Exception as e:
        print(f"Error durante el procesamiento: {str(e)}")
    finally:
        if 'client' in locals():
            client.close()
```

Datos cargados exitosamente

Contenido de la colección 'sales':

```
{'_id': ObjectId('67abf7c174c74bf63f07f76b'), 'region': 'North', 'product': 'Product_D', 'date': d
{'_id': ObjectId('67abf7c174c74bf63f07f76c'), 'region': 'West', 'product': 'Product_C', 'date': da
{'_id': ObjectId('67abf7c174c74bf63f07f76d'), 'region': 'East', 'product': 'Product_C', 'date': da
```

Estadísticas por región:

```
{'_id': 'East', 'total_sales': 296780.0, 'avg_sales': 1262.8936170212767, 'sales_count': 235, 'pro
{'_id': 'West', 'total_sales': 313536.0, 'avg_sales': 1259.1807228915663, 'sales_count': 249, 'pro
{'_id': 'South', 'total_sales': 351882.0, 'avg_sales': 1243.399293286219, 'sales_count': 283, 'pro
{'_id': 'North', 'total_sales': 294153.0, 'avg_sales': 1262.4592274678112, 'sales_count': 233, 'pro
```

Estadísticas por producto:

```
{'_id': 'Product_B', 'total_sales': 286188.0, 'regions_sold': ['South', 'East', 'West', 'North'],
{'_id': 'Product_A', 'total_sales': 275695.0, 'regions_sold': ['North', 'South', 'East', 'West'],
{'_id': 'Product_D', 'total_sales': 335633.0, 'regions_sold': ['South', 'East', 'West', 'North'],
{'_id': 'Product_C', 'total_sales': 358835.0, 'regions_sold': ['South', 'East', 'West', 'North'],
```

Tipos de datos en el primer documento:

```
_id: <class 'bson.objectid.ObjectId'>
region: <class 'str'>
product: <class 'str'>
date: <class 'datetime.datetime'>
sales: <class 'float'>
year_month: <class 'str'>
```

6 Benchmark MongoDB

```
from pyspark.sql import SparkSession
from pymongo import MongoClient
from datetime import datetime, timedelta
import pandas as pd
import time
import statistics
import json
from tabulate import tabulate
import matplotlib.pyplot as plt
import seaborn as sns

class MongoBenchmark:
    def __init__(self, client):
        self.db = client['sales_db']
        self.results = []

    def time_operation(self, operation_name, operation, iterations=100):
        """Mide el tiempo de una operación específica"""
        times = []
        for _ in range(iterations):
            start_time = time.perf_counter()
            operation()
            end_time = time.perf_counter()
            times.append(end_time - start_time)

        metrics = {
            'operation': operation_name,
            'avg_time': statistics.mean(times),
            'min_time': min(times),
            'max_time': max(times),
```

```
        'median_time': statistics.median(times),
        'std_dev': statistics.stdev(times) if len(times) > 1 else 0
    }
    self.results.append(metrics)
    return metrics

def run_benchmarks(self):
    """Ejecuta todos los benchmarks"""
    print("Iniciando benchmarks...")

    # Benchmark 1: Búsqueda simple por región
    self.time_operation(
        "Búsqueda por región",
        lambda: list(self.db.sales.find({"region": "North"}).limit(10))
    )

    # Benchmark 2: Agregación de ventas por región
    self.time_operation(
        "Agregación por región",
        lambda: list(self.db.sales.aggregate([
            {"$group": {
                "_id": "$region",
                "total_sales": {"$sum": "$sales"}
            }}
        ]))
    )

    # Benchmark 3: Búsqueda con filtro de fecha
    self.time_operation(
        "Búsqueda por fecha",
        lambda: list(self.db.sales.find({
            "date": {
                "$gte": datetime(2025, 1, 1),
                "$lt": datetime(2025, 2, 1)
            }
        })))
    )

    # Benchmark 4: Búsqueda compuesta
    self.time_operation(
        "Búsqueda compuesta",
        lambda: list(self.db.sales.find({
            "region": "North",
            "sales": {"$gt": 1000}
        })))
    )

    # Benchmark 5: Agregación compleja
    self.time_operation(
        "Agregación compleja",
        lambda: list(self.db.sales.aggregate([
            {"$group": {
                "_id": {
                    "region": "$region",
                    "product": "$product"
                },
                "total_sales": {"$sum": "$sales"},
                "count": {"$sum": 1}
            }}
        ]))
    )
```

```

        }},
        {"$sort": {"total_sales": -1}},
        {"$limit": 5}
    ]))
)

def generate_report(self):
    """Genera un reporte detallado de los benchmarks"""
    print("\nReporte de Benchmark:")

    # Crear tabla con resultados
    headers = ['Operación', 'Tiempo Promedio (s)', 'Tiempo Min (s)',
               'Tiempo Max (s)', 'Mediana (s)', 'Desv. Estándar']
    table_data = [[
        r['operation'],
        f"{r['avg_time']:.6f}",
        f"{r['min_time']:.6f}",
        f"{r['max_time']:.6f}",
        f"{r['median_time']:.6f}",
        f"{r['std_dev']:.6f}"
    ] for r in self.results]

    print(tabulate(table_data, headers=headers, tablefmt='grid'))

    # Generar gráfico de barras
    operations = [r['operation'] for r in self.results]
    avg_times = [r['avg_time'] for r in self.results]

    plt.figure(figsize=(12, 6))
    sns.barplot(x=operations, y=avg_times)
    plt.xticks(rotation=45, ha='right')
    plt.xlabel('Operación')
    plt.ylabel('Tiempo Promedio (segundos)')
    plt.title('Rendimiento de Operaciones MongoDB')
    plt.tight_layout()
    plt.savefig('mongodb_benchmark.png')

    # Guardar resultados en JSON
    with open('benchmark_results.json', 'w') as f:
        json.dump(self.results, f, indent=2)

def run_complete_benchmark():
    """Ejecuta el benchmark completo"""
    try:
        client = MongoClient('mongodb://localhost:27017/')
        benchmark = MongoBenchmark(client)

        print("Iniciando suite de benchmarks...")
        start_time = time.time()

        benchmark.run_benchmarks()
        benchmark.generate_report()

        total_time = time.time() - start_time
        print(f"\nTiempo total de ejecución: {total_time:.2f} segundos")

    except Exception as e:
        print(f"Error durante el benchmark: {str(e)}")

```

```
finally:
    client.close()

if __name__ == "__main__":
    run_complete_benchmark()
```

Iniciando suite de benchmarks...
Iniciando benchmarks...

Reporte de Benchmark:

Operación	Tiempo Promedio (s)	Tiempo Min (s)	Tiempo Max (s)	Mediana
Búsqueda por región	0.00015	0.00012	0.001381	0.00015
Agregación por región	0.000378	0.00034	0.000701	0.000378
Búsqueda por fecha	0.001942	0.001371	0.046697	0.001942
Búsqueda compuesta	0.000478	0.000427	0.000789	0.000478
Agregación compleja	0.000456	0.000404	0.000712	0.000456

Tiempo total de ejecución: 0.57 segundos

